

By: Shoumik, Moneba, Judy, Ekam & Abrar

Group Project: Team AB
FreshRoute Schedule.pdf

Team Members:

Name	Email	Student ID
Moneba Hanifi	moneba@my.yorku.ca	219721810
Shoumik Ahmed	shoumik@my.yorku.ca	219881598
Judy El Ghargomi	jelg21@my.yorku.ca	220227237
Ekambir Momi	ekam727@my.yorku.ca	219424407
Abrar Jawad Tarafder	abrart03@my.yorku.ca	219453497

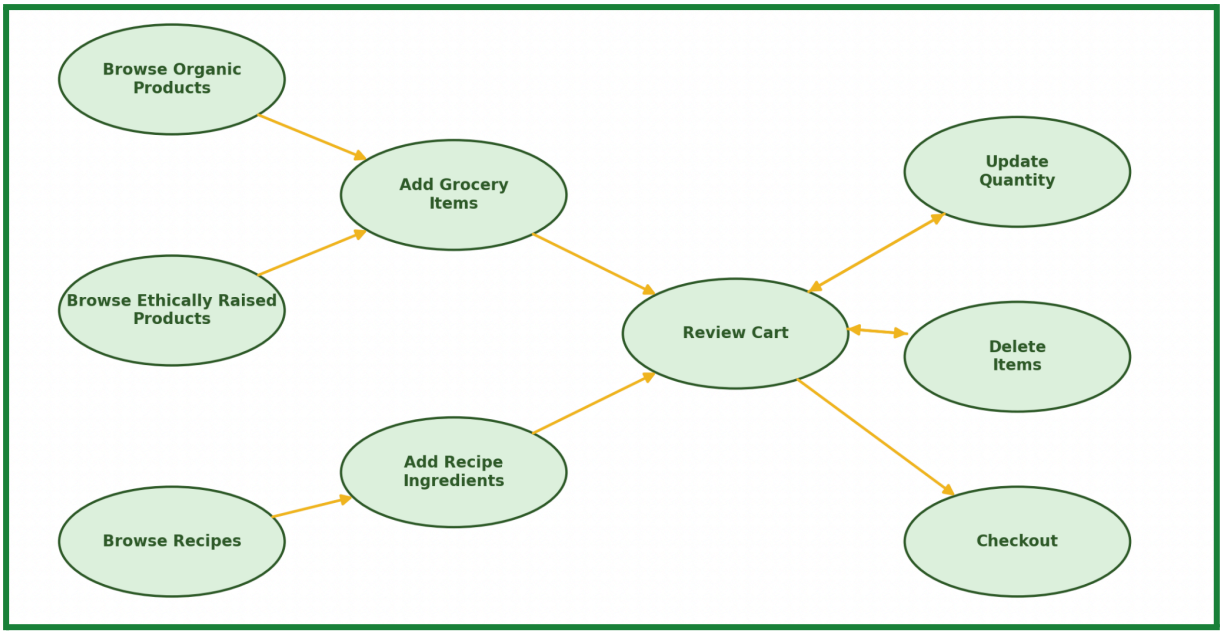
3311 Section M Term W

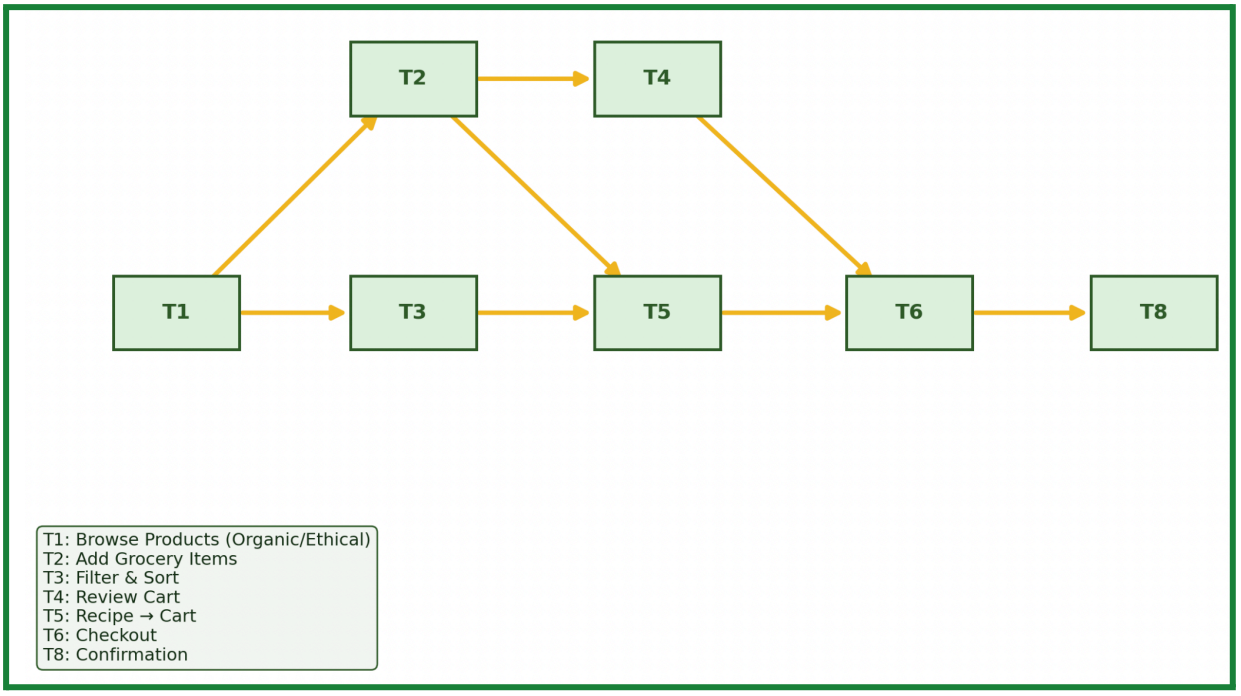
Class Instructor: Ilir Dema

Sprint 2 Duration: March 10 - March 23, 2026

Network Diagram

The visual representation of task dependencies is attached below.





The **top diagram** shows the user workflow of the FreshRoute system, illustrating how different user actions lead to others. For example, users can browse organic or ethically raised products to add grocery items, review their cart, update quantities, delete items, or proceed to checkout. It focuses on how users interact with the system and how features are connected.

The **bottom diagram** represents the task dependency network for development. It shows the order in which system tasks must be implemented, such as browsing products before adding items, and completing checkout before confirmation. This diagram focuses on the logical sequence and dependencies among the development tasks required to build the overall system, regardless of the sprint.

Critical Path

The critical path determines the main dependency chain that has the greatest impact on whether the sprint can be completed on time. For FreshRoute Sprint 2, the critical path was:

1. **Browse Organic Products / Browse Ethically Raised Products**
2. **Add Grocery Items**
3. **Review Cart**
4. **Checkout**

This path was treated as the most important sequence because it represents the core shopping flow of the application. A user must first browse products, then add items to the cart, review the cart contents, and finally proceed to checkout. The recipe flow was also important, but it acted as a supporting branch that fed into the cart rather than replacing the main checkout path. After checkout is completed, the system will generate an order confirmation (with potential tracking), which will be implemented as the final step following the checkout process in Sprint 03.

Strategies to Stay on Schedule

Task Prioritization: We first focused on building the main product-related pages, including Organic, Ethically Raised, and Recipes. After those core pages were completed, we moved on to the cart-related functionality, including adding items to the cart and developing the cart page. This helped us build the main shopping flow in a clear, ordered way. For Sprint 2, we also ensured that at least 30% of the Sprint's GitHub commits were completed within the first week.

Parallel Development: Different team members worked on filtering, recipe functionality, organic and ethical pages, cart features, documentation, database population, and backend integration simultaneously, enabling progress across multiple areas of the sprint.

GitHub Branching: We used separate feature branches for cart updates, recipe-related work, filtering, organic and ethical features and backend integration so changes could be developed independently and merged in a more organized way.

Standups and Check-ins: Regular standups helped the team identify blockers early, communicate progress, and reassign effort when needed.

Incremental Integration: We built the user-facing pages first and then connected supporting backend and data integration work step by step, which helped reduce risk during the sprint

What Went Wrong

One challenge during the sprint was that several tasks depended on shared cart behaviour and integration logic. Because both grocery products and recipe ingredients eventually made their way into the cart, changes in one part of the system sometimes affected another, requiring extra testing. Another issue was that backend integration required more coordination than expected, especially when connecting product data, recipe data, and images with the frontend pages. This slowed some parts of the workflow and made debugging more time-consuming than initially planned.

Lessons Learned

Identify the main user flow early: Defining the cart and checkout flow as the most important path helped the team protect the most valuable work during the sprint.

Start integration-heavy work sooner: Features that depend on shared data or shared cart logic should begin earlier to allow more time for testing and debugging.

Keep tasks at the right level of detail: For schedule planning, it is better to focus on major dependent tasks, such as browsing, adding items, reviewing the cart, and checkout, rather than low-level implementation details.

Parallel work must still be coordinated carefully: Even when team members work on different branches, related features still require frequent communication to ensure smoother integration later.